Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Operating Systems – ENCS3390

Project Report

Prepared by:

Name: Yasmin Al Shawawrh. **Number:** 1220848.

Instructor: Abdel Salam Sayyad.

Date: 8/12/2024.

Abstract

This project explores the performance of different parallel processing techniques for counting word frequencies in a large text dataset, and identifying the top 10 most frequent words. The project compares three approaches: a naive approach, where all tasks are executed sequentially without parallelism; a multiprocessing approach, which utilizes multiple child processes to run tasks concurrently; and a multithreading approach, which leverages multiple threads for parallel execution. The performance of each approach is evaluated by measuring the execution time and analyzing the impact of varying the number of processes and threads (2, 4, 6, and 8). Additionally, the project applies Amdahl's Law to analyze the theoretical speedup and determine the optimal number of processes or threads based on the available cores. The results are presented in a comparative table, followed by a discussion on the differences in performance and the effectiveness of parallelism in speeding up word frequency computation. The findings offer insights into the trade-offs between simplicity, efficiency, and parallel computing strategies in processing large datasets.

Table of Contents

Abstract II
Table of figures IV
List of tables V
Theory 1
- **Naïve Approach** 1
- **Multiprocessing Approach** 1
- **Multithreading Approach** 1

Discussion and Analysis 2
- **Naïve Approach** 2
- **Multiprocessing Approach** 6
- **Multithreading Approach** 12

Project Questions 18
Conclusion 23

Table of figures

FIGURE 1: NAÏVE APPROACH CODE. 4
FIGURE 2: NAÏVE APPROACH OUTPUT. 4
FIGURE 3: MULTIPROCESSING APPROACH CODE. 9
FIGURE 4: MULTIPROCESSING APPROACH OUTPUT FOR (2 CHILDS). 9
FIGURE 5: MULTIPROCESSING APPROACH OUTPUT FOR (4 CHILDS). 9
FIGURE 6: MULTIPROCESSING APPROACH OUTPUT FOR (6 CHILDS). 10
FIGURE 7: MULTIPROCESSING APPROACH OUTPUT FOR (8 CHILDS). 10
FIGURE8 : MULTITHREADING APPROACH CODE. 14
FIGURE 9: MULTITHREADIND APPROACH OUTPUT FOR (2 THREADS). 15
FIGURE 10: MULTITHREADIND APPROACH OUTPUT FOR (4 THREADS). 15
FIGURE 11: MULTITHREADIND APPROACH OUTPUT FOR (6 THREADS). 15
FIGURE 12: MULTITHREADIND APPROACH OUTPUT FOR (8 THREADS). 16

List of tables

TABLE 1: MULTIPROCESSING APPROACH TABLE. 11
TABLE 2: MULTITHREADING APPROACH TABLE. 17
TABLE 3: THREE APPROACHES TABLE. 22

Theory

- **Naïve Approach**

The naive approach is a basic and simple method of solving a problem. It involves performing tasks one step at a time, without using any advanced techniques or trying to make the process faster. While it is easy to understand and implement, this approach can be slow and inefficient, especially for larger or more complex problems. It doesn't use multiple resources or try to improve speed, making it a good starting point to compare with more advanced methods like using multiple processors or optimizing the process. Despite its limitations, the naive approach can be useful for small problems where speed isn't as important.

- **Multiprocessing Approach**

The multiprocessing approach refers to using multiple processes to perform tasks simultaneously, allowing a program to take advantage of multiple CPU cores. Each process runs independently and can perform a specific part of a larger task, helping to speed up the overall execution. This approach is especially useful for tasks that require a lot of computation, as it divides the work among several processes, reducing the time needed to complete the task. However, it also requires proper management of processes to ensure they work together efficiently.

- **Multithreading Approach**

The multithreading approach involves using multiple threads within a single process to execute tasks concurrently. Each thread can perform a specific part of the task, allowing the program to handle multiple operations at the same time. This is particularly useful for programs that need to perform tasks like handling multiple user inputs, performing background operations, or processing data in parallel. Unlike multiprocessing, where each process runs in its own memory space, threads share the same memory space within a process, making communication between them easier and more efficient. However, this also requires careful management to avoid conflicts or errors when multiple threads access the same data.

Discussion and Analysis

- **Naïve Approach**

➢ **Code**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <unistd.h>

#define MAX_WORD_LENGTH 120
#define MAX_WORDS 18000000
#define MAX_FREQ_WORDS 260000

typedef struct {
    char *word;
    int frequency;
} WordFrequency;

// Function to read the file and store words
char **read_file(char *filename, int *count) {
    char buffer[MAX_WORD_LENGTH];
    *count = 0;
    char **words = malloc(MAX_WORDS * sizeof(char *));
    if (words == NULL) {
        printf("Error: Memory allocation for words array failed.\n");
        return NULL;
    }

    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        printf("File not found: %s.\n", filename);
        free(words);
        return NULL;
    }
```

```c
    while (fscanf(file, "%s", buffer) != EOF && *count < MAX_WORDS) {
        if (strlen(buffer) > 0 && strlen(buffer) < MAX_WORD_LENGTH) {
            words[*count] = malloc(strlen(buffer) + 1);
            if (words[*count] == NULL) {
                printf("Error: Memory allocation failed for word %d.\n", *count);
                break;
            }
            strcpy(words[*count], buffer);
            (*count)++;
        }
    }

    fclose(file);

    if (*count == 0) {
        printf("No words were read from the file.\n");
        free(words);
        return NULL;
    }

    return words;
}

int compare_words(const void *a, const void *b) {
    return strcmp(*(const char **)a, *(const char **)b);
}

WordFrequency *create_frequency_array(char **words, int word_count, int *freq_count) {
    WordFrequency *freq_array = malloc(MAX_FREQ_WORDS * sizeof(WordFrequency));
    if (freq_array == NULL) {
        printf("Error: Memory allocation for frequency array failed.\n");
```

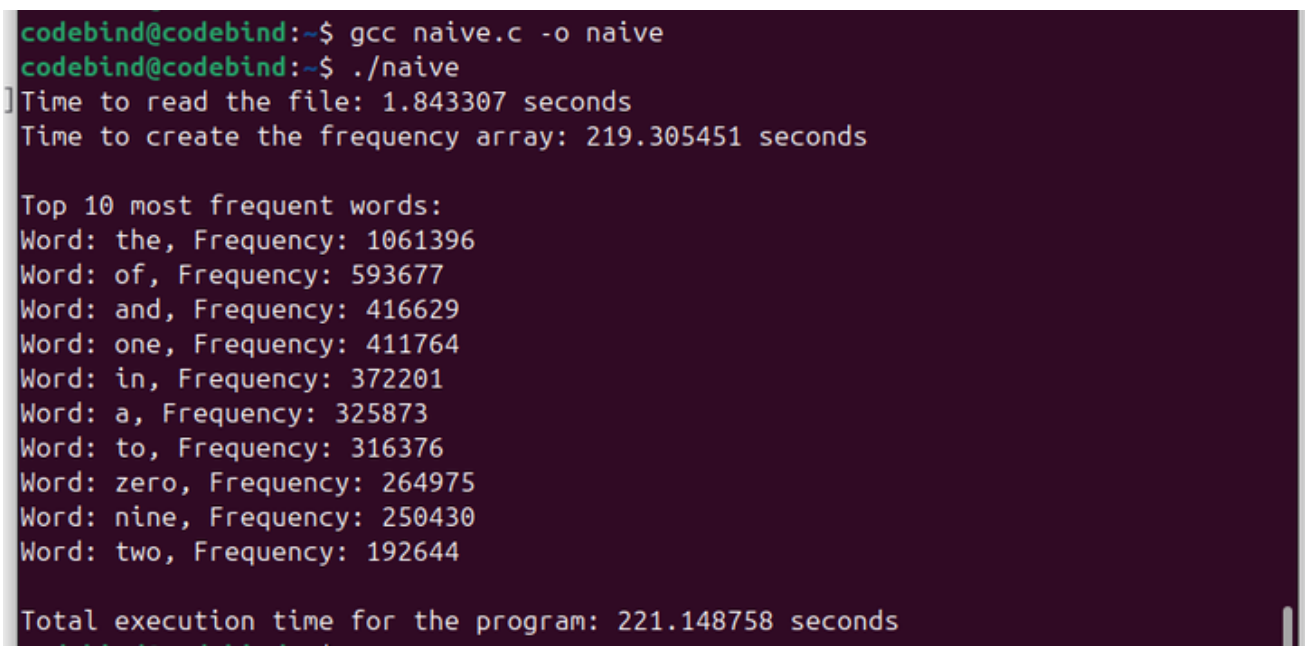
```c
    if (freq_array == NULL) {
        printf("Error: Memory allocation for frequency array failed.\n");
        return NULL;
    }

    qsort(words, word_count, sizeof(char *), compare_words);

    *freq_count = 0;

    for (int i = 0; i < word_count; i++) {
        if (*freq_count == 0 || strcmp(words[i], freq_array[*freq_count - 1].word) != 0) {
            if (*freq_count < MAX_FREQ_WORDS) {
                freq_array[*freq_count].word = words[i];
                freq_array[*freq_count].frequency = 1;
                (*freq_count)++;
            }
        } else {
            freq_array[*freq_count - 1].frequency++;
        }
    }

    return freq_array;
}

// Function to compare frequencies of words
int compare_frequency(const void *a, const void *b) {
    WordFrequency *wordA = (WordFrequency *)a;
    WordFrequency *wordB = (WordFrequency *)b;
    return wordB->frequency - wordA->frequency;
}

// Function to free memory allocated for words
```

```c
// Function to free memory allocated for words
void free_words(char **words, int count) {
    for (int i = 0; i < count; i++) {
        free(words[i]);
    }
    free(words);
}

int main() {
    struct timeval start, end;
    double elapsed_read, elapsed_freq, elapsed_sort, elapsed_total, total_time;
    gettimeofday(&start, NULL);
    char *filename = "large_file.txt";
    int word_count = 0;
    gettimeofday(&start, NULL);
    char **words = read_file(filename, &word_count);
    gettimeofday(&end, NULL);
    elapsed_read = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time to read the file: %f seconds\n", elapsed_read);

    if (words != NULL) {
        int freq_count = 0;
        gettimeofday(&start, NULL);
        WordFrequency *freq_array = create_frequency_array(words, word_count, &freq_count);
        gettimeofday(&end, NULL);
        elapsed_freq = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
        printf("Time to create the frequency array: %f seconds\n", elapsed_freq);

        if (freq_array != NULL) {
            // Measure time to sort the frequency array
            gettimeofday(&start, NULL);
```

```c
    WordFrequency *freq_array = create_frequency_array(words, word_count, &freq_count);
    gettimeofday(&end, NULL);
    elapsed_freq = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time to create the frequency array: %f seconds\n", elapsed_freq);

    if (freq_array != NULL) {
        // Measure time to sort the frequency array
        gettimeofday(&start, NULL);
        qsort(freq_array, freq_count, sizeof(WordFrequency), compare_frequency);
        gettimeofday(&end, NULL);
        elapsed_sort = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
        printf("Time to sort the frequency array: %f seconds\n", elapsed_sort);

        // Display the top 10 most frequent words
        printf("\nTop 10 most frequent words:\n");
        for (int i = 0; i < 10 && i < freq_count; i++) {
            printf("Word: %s, Frequency: %d\n", freq_array[i].word, freq_array[i].frequency);
        }

        free(freq_array);
    }

    free_words(words, word_count);
}
gettimeofday(&end, NULL);
elapsed_total = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
total_time = elapsed_read + elapsed_freq + elapsed_sort;
printf("\nTotal execution time for the program: %f seconds\n", total_time);

return 0;
}
```

Figure 1: Naïve Approach Code.

- **Output**



```
codebind@codebind:~$ gcc naive.c -o naive
codebind@codebind:~$ ./naive
Time to read the file: 1.843307 seconds
Time to create the frequency array: 219.305451 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: and, Frequency: 416629
Word: one, Frequency: 411764
Word: in, Frequency: 372201
Word: a, Frequency: 325873
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644

Total execution time for the program: 221.148758 seconds
```

Figure 2: Naïve Approach output.

- **Analysis**

The program reads a text file (in this case, large_file.txt) and identifies the top 10 most frequent words by employing a naive approach, which does not involve any child processes or threads. The program begins by reading the file into an array of words using the read_file function. It uses the fscanf function to read each word and stores it dynamically in memory. After the file is read, the words are sorted using qsort, and a frequency array is created to count how many times each word appears using the create_frequency_array function. The program then sorts this frequency array in descending order of word frequencies. For performance measurement, the gettimeofday function is used to track the time taken to read the file, create the frequency array, and sort it. The program prints the top 10 most frequent words, along with their frequencies, and also measures the total execution time. The approach is considered naive because it processes the data sequentially without the use of parallelism.

$$\textit{Performance (Throughput)} = \frac{1}{Time}$$

The naive approach in this program calculates the top 10 most frequent words without using any parallelization, meaning it processes the task sequentially, one step at a time. While this method may not be the fastest, it serves as a basic building block for future optimizations. By measuring performance through throughput (1/time), we can see that while the execution time might be higher, this approach establishes the groundwork for more advanced techniques, such as parallelization, which can significantly improve speed and resource utilization in subsequent iterations of the program.

- **Multiprocessing Approach**

➢ **Code**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/time.h>

#define MAX_WORD_LENGTH 120
#define MAX_WORDS 18000000
#define MAX_FREQ_WORDS 260000

typedef struct
{
    char *word;
    int frequency;
} WordFrequency;

// Function to read the file and store words
char **read_file(char *filename, int *count)
{
    char buffer[MAX_WORD_LENGTH];
    *count = 0;
    char **words = malloc(MAX_WORDS * sizeof(char *));
    if (words == NULL)
    {
        printf("Error: Memory allocation for words array failed.\n");
        return NULL;
    }

    FILE *file = fopen(filename, "r");
    if (file == NULL)
```

```c
    if (file == NULL)
    {
        printf("File not found: %s.\n", filename);
        free(words);
        return NULL;
    }

    while (fscanf(file, "%s", buffer) != EOF && *count < MAX_WORDS)
    {
        if (strlen(buffer) > 0 && strlen(buffer) < MAX_WORD_LENGTH)
        {
            words[*count] = malloc(strlen(buffer) + 1);
            if (words[*count] == NULL)
            {
                printf("Error: Memory allocation failed for word %d.\n", *count);
                break;
            }
            strcpy(words[*count], buffer);
            (*count)++;
        }
    }

    fclose(file);
    return words;
}

// Function to compare words for sorting
int compare_words(const void *a, const void *b)
{
    return strcmp(*(const char **)a, *(const char **)b);
}

void process_chunk(char **words, int start, int end, FILE *temp_file)
```

Dec 8 17:30

Open multiprocessing.c

naive.c | multiprocessing.c | multithreading.c

```c
void free_words(char **words, int count)
{
    for (int i = 0; i < count; i++)
    {
        free(words[i]);
    }
    free(words);
}
int main()
{
    struct timeval start, end;
    double elapsed;
    gettimeofday(&start, NULL);
    int num_processes = 8; // Use 2 processes or 4 or 6 or 8
    pid_t pids[num_processes];
    char *filename = "large_file.txt";
    int word_count = 0;
    gettimeofday(&start, NULL);
    char **words = read_file(filename, &word_count);
    gettimeofday(&end, NULL);
    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time taken to read the file: %f seconds\n", elapsed);
    if (words == NULL) return 1;
    gettimeofday(&start, NULL);
    qsort(words, word_count, sizeof(char *), compare_words);
    gettimeofday(&end, NULL);
    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time taken to sort the words: %f seconds\n", elapsed);
    int chunk_size = word_count / num_processes;
    for (int i = 0; i < num_processes; i++)
    {
        pids[i] = fork();
        if (pids[i] < 0)
```

Dec 8 17:30

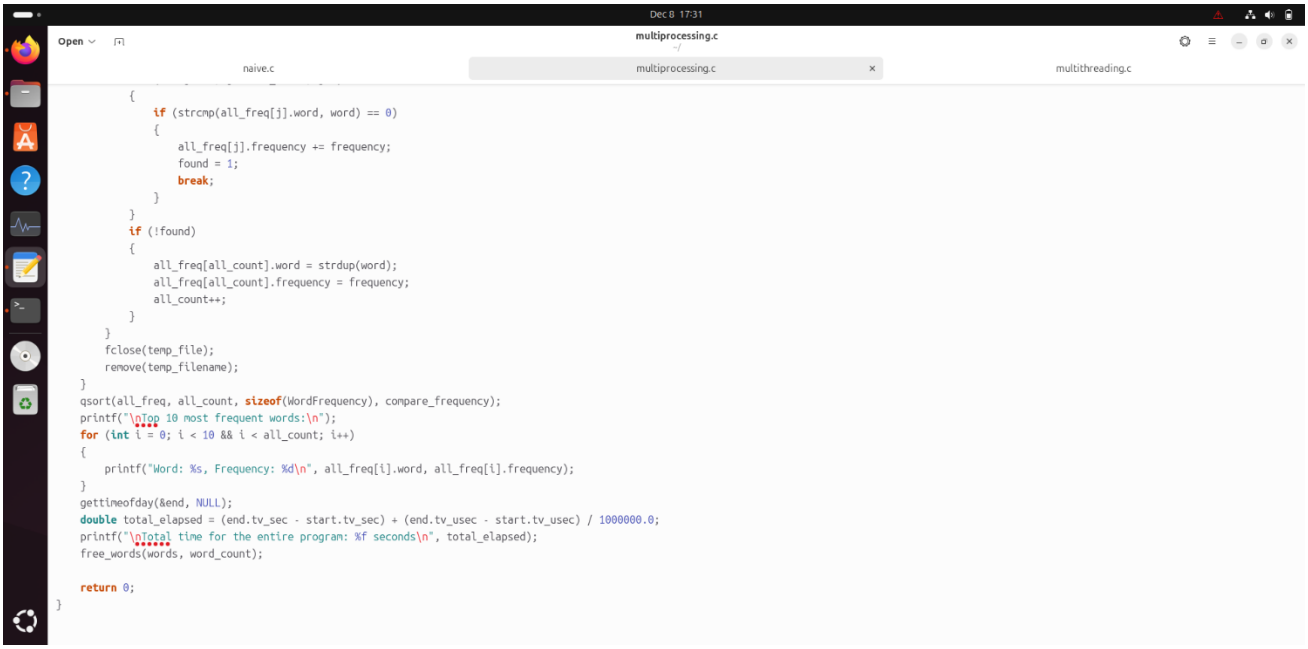
Open multiprocessing.c

naive.c | multiprocessing.c | multithreading.c

```c
void process_chunk(char **words, int start, int end, FILE *temp_file)
{
    WordFrequency freq_array[MAX_FREQ_WORDS];
    int count = 0;
    for (int i = start; i < end; i++)
    {
        if (count == 0 || strcmp(words[i], freq_array[count - 1].word) != 0)
        {
            freq_array[count].word = words[i];
            freq_array[count].frequency = 1;
            count++;
        }
        else
        {
            freq_array[count - 1].frequency++;
        }
    }
    for (int i = 0; i < count; i++)
    {
        fprintf(temp_file, "%s %d\n", freq_array[i].word, freq_array[i].frequency);
    }
}

// Function to compare frequencies of words
int compare_frequency(const void *a, const void *b)
{
    WordFrequency *wordA = (WordFrequency *)a;
    WordFrequency *wordB = (WordFrequency *)b;
    return wordB->frequency - wordA->frequency;
}

// Function to free memory allocated for words
void free_words(char **words, int count)
```

```c
    pids[i] = fork();
    if (pids[i] < 0)
    {
        perror("Fork failed");
        return 1;
    }
    if (pids[i] == 0)
    {
        char temp_filename[50];
        snprintf(temp_filename, sizeof(temp_filename), "temp_file_%d.txt", i);
        FILE *temp_file = fopen(temp_filename, "w");
        if (temp_file == NULL)
        {
            perror("Error creating temp file");
            exit(1);
        }
        int start_idx = i * chunk_size;
        int end_idx;
        if (i == num_processes - 1)
        {
            end_idx = word_count;
        }
        else
        {
            end_idx = (i + 1) * chunk_size;
        }

        process_chunk(words, start_idx, end_idx, temp_file);
        fclose(temp_file);
        exit(0);
    }
}
// Wait for all child processes to finish
```

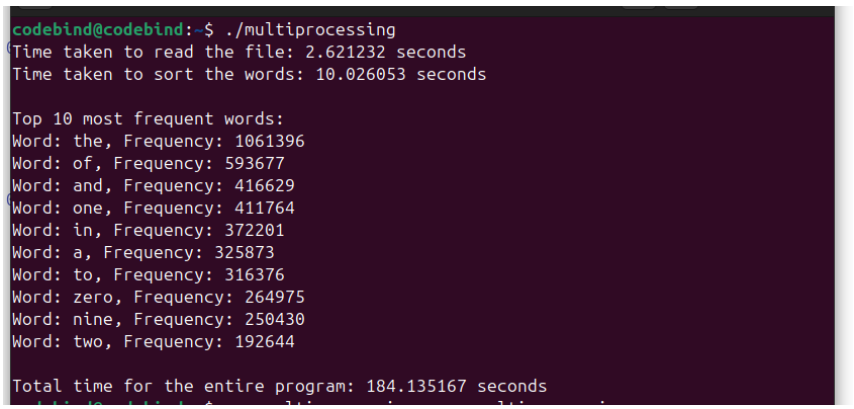
```c
// Wait for all child processes to finish
for (int i = 0; i < num_processes; i++)
{
    waitpid(pids[i], NULL, 0);
}
// Combine results from temporary files
FILE *final_file = fopen("final_output.txt", "w");
if (final_file == NULL)
{
    perror("Error creating final output file");
    return 1;
}
WordFrequency all_freq[MAX_FREQ_WORDS];
int all_count = 0;
for (int i = 0; i < num_processes; i++)
{
    char temp_filename[50];
    snprintf(temp_filename, sizeof(temp_filename), "temp_file_%d.txt", i);

    FILE *temp_file = fopen(temp_filename, "r");
    if (temp_file == NULL)
    {
        perror("Error reading temp file");
        continue;
    }

    char word[MAX_WORD_LENGTH];
    int frequency;
    while (fscanf(temp_file, "%s %d", word, &frequency) != EOF)
    {
        int found = 0;
        for (int j = 0; j < all_count; j++)
        {
```



```
            {
                if (strcmp(all_freq[j].word, word) == 0)
                {
                    all_freq[j].frequency += frequency;
                    found = 1;
                    break;
                }
            }
            if (!found)
            {
                all_freq[all_count].word = strdup(word);
                all_freq[all_count].frequency = frequency;
                all_count++;
            }
        }
        fclose(temp_file);
        remove(temp_filename);
    }
    qsort(all_freq, all_count, sizeof(WordFrequency), compare_frequency);
    printf("\nTop 10 most frequent words:\n");
    for (int i = 0; i < 10 && i < all_count; i++)
    {
        printf("Word: %s, Frequency: %d\n", all_freq[i].word, all_freq[i].frequency);
    }
    gettimeofday(&end, NULL);
    double total_elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("\nTotal time for the entire program: %f seconds\n", total_elapsed);
    free_words(words, word_count);

    return 0;
}
```

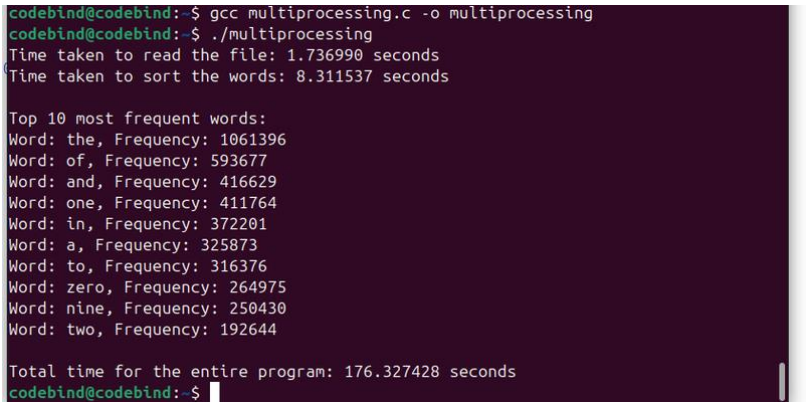
Figure 3: Multiprocessing Approach Code.

- **Output**



```
codebind@codebind:~$ ./multiprocessing
Time taken to read the file: 2.621232 seconds
Time taken to sort the words: 10.026053 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: and, Frequency: 416629
Word: one, Frequency: 411764
Word: in, Frequency: 372201
Word: a, Frequency: 325873
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644

Total time for the entire program: 184.135167 seconds
```

Figure 4: Multiprocessing Approach output for (2 childs).



```
codebind@codebind:~$ gcc multiprocessing.c -o multiprocessing
codebind@codebind:~$ ./multiprocessing
Time taken to read the file: 1.736990 seconds
Time taken to sort the words: 8.311537 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: and, Frequency: 416629
Word: one, Frequency: 411764
Word: in, Frequency: 372201
Word: a, Frequency: 325873
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644

Total time for the entire program: 176.327428 seconds
codebind@codebind:~$
```

Figure 5: Multiprocessing Approach output for (4 childs).

```
codebind@codebind:~$ gcc multiprocessing.c -o multiprocessing
codebind@codebind:~$ ./multiprocessing
Time taken to read the file: 1.422729 seconds
Time taken to sort the words: 6.975206 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: and, Frequency: 416629
Word: one, Frequency: 411764
Word: in, Frequency: 372201
Word: a, Frequency: 325873
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644

Total time for the entire program: 129.949859 seconds
codebind@codebind:~$
```

Figure 6: Multiprocessing Approach output for (6 childs).

```
codebind@codebind:~$ gcc multiprocessing.c -o multiprocessing
codebind@codebind:~$ ./multiprocessing
Time taken to read the file: 1.832383 seconds
Time taken to sort the words: 8.230930 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: and, Frequency: 416629
Word: one, Frequency: 411764
Word: in, Frequency: 372201
Word: a, Frequency: 325873
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644

Total time for the entire program: 174.948847 seconds
codebind@codebind:~$
```

Figure 7: Multiprocessing Approach output for (8 childs).

- **Analysis**

This program efficiently calculates word frequencies from a large text file by leveraging parallel processing. By dividing the workload across multiple processes and merging their results, it optimizes execution time for large datasets. Each child process operates independently in separate memory spaces, necessitating a final merging step to combine their results into a cohesive output. This approach is especially useful for processing extensive datasets on multicore systems.

The program begins by reading the input text file using the read_file function. Each word is stored dynamically in memory, allowing flexibility for varying word lengths. After reading the words, the program uses the qsort function to alphabetically sort the array. Sorting ensures that identical words are grouped together, simplifying the frequency calculation in subsequent steps.

Creating Child Processes and Dividing Work: to take advantage of parallel processing, the program divides the sorted word array into equal chunks, with each chunk assigned to a separate child process. The fork system call creates these processes, and each operates in its own memory space. This isolation ensures that each process can safely calculate word frequencies within its assigned range using the process_chunk function. The results are written to separate temporary files for each process. While this design avoids memory conflicts, the independent memory spaces of the child processes require a merging step to consolidate the results later.

Merging Results from Child Processes: fter all child processes complete their tasks, the parent process combines their outputs. Using the waitpid function, the parent ensures that all child processes finish before proceeding. The temporary files generated by each process are then read, and the word-frequency pairs are combined. If a word from a temporary file matches an existing word in the global frequency array, their frequencies are aggregated. Otherwise, a new entry is created. This merging step is essential since each process worked in isolated memory spaces, and without it, the results would remain fragmented.

Once the results are merged, the program sorts the global frequency array in descending order of frequency using qsort. This allows the most frequent words to appear first in the output. The program then displays the top 10 most frequent words, demonstrating its ability to process large text files and extract meaningful insights.

- **Different Number of processes**

Number of processes	**Execution Time**	**Performance**
2 processes	196.78254sec	5.08175166ms-1
4 processes	186.3755955sec	5.365498 ms-1
6 processes	138.347824sec	7.228155 ms-1
8 processes	185.01216sec	5.40505 ms-1

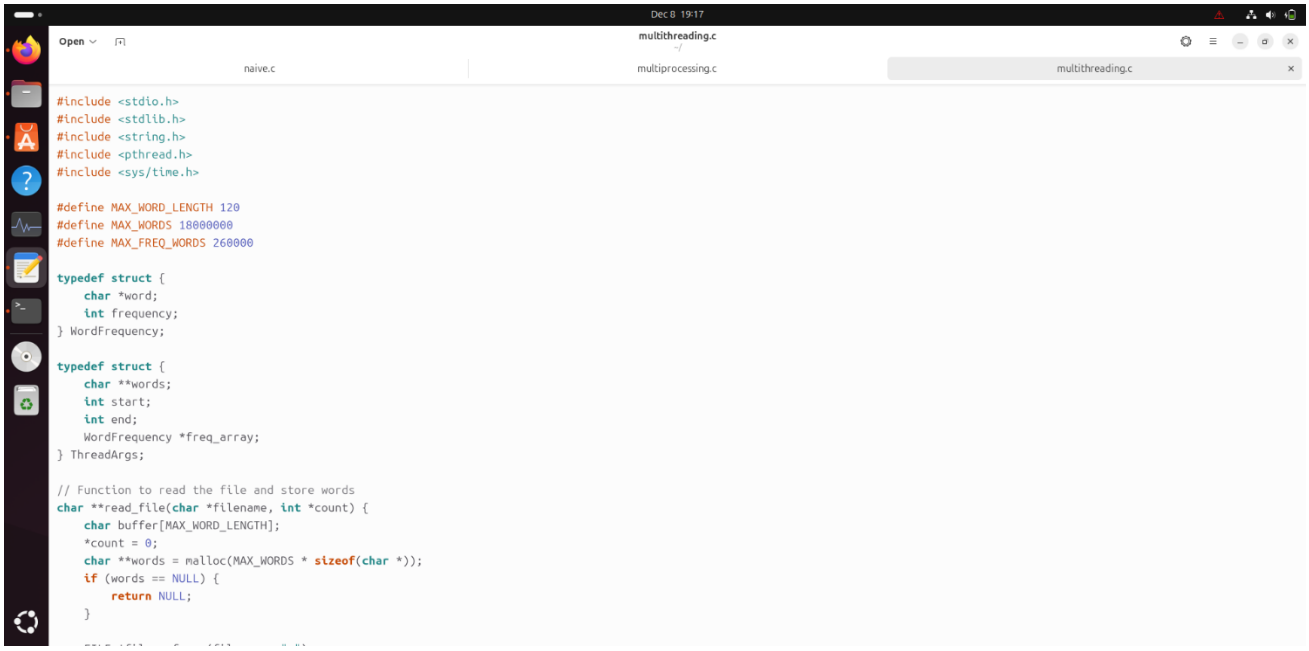
Table 1: Multiprocessing Approach table.

Why does the execution time change when using different numbers of child processes in parallel?

As the number of processes increases, the execution time typically decreases because the workload is divided among multiple processes, allowing for parallel execution on available CPU cores, which improves performance. However, once the number of processes exceeds the number of available CPU cores, the system starts to experience overhead due to context switching, where the CPU has to switch between processes, causing a decrease in efficiency. As shown in our outputs.

- **Multithreading Approach**

➢ **Code**



```c
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        free(words);
        return NULL;
    }

    while (fscanf(file, "%s", buffer) != EOF && *count < MAX_WORDS) {
        if (strlen(buffer) > 0 && strlen(buffer) < MAX_WORD_LENGTH) {
            words[*count] = malloc(strlen(buffer) + 1);
            if (words[*count] == NULL) {
                break;
            }
            strcpy(words[*count], buffer);
            (*count)++;
        }
    }

    fclose(file);
    return words;
}

// Function to compare words for sorting
int compare_words(const void *a, const void *b) {
    return strcmp(*(const char **)a, *(const char **)b);
}

// Function to process chunks of words in each thread
void *process_chunk(void *args) {
    ThreadArgs *data = (ThreadArgs *)args;
    char **words = data->words;
    int start_idx = data->start;
```

```c
    int start_idx = data->start;
    int end_idx = data->end;
    WordFrequency *freq_array = data->freq_array;

    int count = 0;
    for (int i = start_idx; i < end_idx; i++) {
        if (count == 0 || strcmp(words[i], freq_array[count - 1].word) != 0) {
            freq_array[count].word = words[i];
            freq_array[count].frequency = 1;
            count++;
        } else {
            freq_array[count - 1].frequency++;
        }
    }

    pthread_exit(NULL);
}

// Function to compare frequencies of words
int compare_frequency(const void *a, const void *b) {
    WordFrequency *wordA = (WordFrequency *)a;
    WordFrequency *wordB = (WordFrequency *)b;
    return wordB->frequency - wordA->frequency;
}

// Function to free memory allocated for words
void free_words(char **words, int count) {
    for (int i = 0; i < count; i++) {
        free(words[i]);
    }
    free(words);
```

```c
    free(words);
}

int main() {
    struct timeval start, end;
    double elapsed;
    gettimeofday(&start, NULL);
    int num_threads=8;// Use 2 threads or 4 or 6 or 8
    pthread_t threads[num_threads];
    char *filename = "large_file.txt";
    int word_count = 0;
    gettimeofday(&start, NULL);
    char **words = read_file(filename, &word_count);
    gettimeofday(&end, NULL);
    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time taken to read the file: %f seconds\n", elapsed);
    if (words == NULL) return 1;
    qsort(words, word_count, sizeof(char *), compare_words);
    WordFrequency freq_array[MAX_FREQ_WORDS];
    memset(freq_array, 0, MAX_FREQ_WORDS * sizeof(WordFrequency));
    int chunk_size = word_count / num_threads;
    gettimeofday(&start, NULL);
    for (int i = 0; i < num_threads; i++) {
        int start_idx = i * chunk_size;
        int end_idx;
        if (i == num_threads - 1)
        {
            end_idx = word_count;
        }
        ThreadArgs *args = malloc(sizeof(ThreadArgs));
        args->words = words;
        args->start = start_idx;
```

```c
            end_idx = word_count;
        }
        ThreadArgs *args = malloc(sizeof(ThreadArgs));
        args->words = words;
        args->start = start_idx;
        args->end = end_idx;
        args->freq_array = freq_array;
        pthread_create(&threads[i], NULL, process_chunk, args);
    }
    gettimeofday(&end, NULL);
    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time taken for thread creation: %f seconds\n", elapsed);
    gettimeofday(&start, NULL);
    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }
    gettimeofday(&end, NULL);
    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("Time taken for threads to process: %f seconds\n", elapsed);
    qsort(freq_array, MAX_FREQ_WORDS, sizeof(WordFrequency), compare_frequency);
    printf("\nTop 10 most frequent words:\n");
    for (int i = 0; i < 10 && freq_array[i].word != NULL; i++) {
        printf("Word: %s, Frequency: %d\n", freq_array[i].word, freq_array[i].frequency);
    }
    gettimeofday(&end, NULL);
    double total_elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0;
    printf("\nTotal time for the entire program: %f seconds\n", total_elapsed);
    free_words(words, word_count);
    return 0;
}
```

Figure8 : multithreading Approach Code.

- **Output**

```
codebind@codebind:~$ gcc multithreading.c -o multithreading -pthread
codebind@codebind:~$ ./multithreading
Time taken to read the file: 1.476504 seconds


Time taken for thread creation: 0.000768 seconds
Time taken for threads to process: 0.425240 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: of, Frequency: 593677
Word: one, Frequency: 411764
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: nine, Frequency: 250430
Word: two, Frequency: 192644
Word: s, Frequency: 116710
Word: three, Frequency: 114775
Word: was, Frequency: 112807

Total time for the entire program: 0.442076 seconds
```

Figure 9: Multithreadind Approach output for (2 threads).

```
codebind@codebind:~$ gcc multithreading.c -o multithreading -pthread
codebind@codebind:~$ ./multithreading


Time taken to read the file: 1.657064 seconds
Time taken for thread creation: 0.000935 seconds
Time taken for threads to process: 0.307287 seconds

Top 10 most frequent words:
Word: the, Frequency: 1061396
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: two, Frequency: 192644
Word: three, Frequency: 114775
Word: was, Frequency: 112807
Word: that, Frequency: 109510
Word: with, Frequency: 95603
Word: this, Frequency: 58832
Word: which, Frequency: 54788

Total time for the entire program: 0.368265 seconds
```

Figure 10: Multithreadind Approach output for (4 threads).

```
codebind@codebind:~$ gcc multithreading.c -o multithreading -pthread
codebind@codebind:~$ ./multithreading
Time taken to read the file: 1.997284 seconds
Time taken for thread creation: 0.000934 seconds
Time taken for threads to process: 0.248271 seconds

Top 10 most frequent words:
Word: to, Frequency: 316376
Word: the, Frequency: 306527
Word: zero, Frequency: 264975
Word: two, Frequency: 192644
Word: three, Frequency: 114775
Word: was, Frequency: 112807
Word: with, Frequency: 95603
Word: this, Frequency: 58832
Word: which, Frequency: 54788
Word: were, Frequency: 39086

Total time for the entire program: 0.311336 seconds
codebind@codebind:~$
```

Figure 11: Multithreadind Approach output for (6 threads).

```
codebind@codebind:~$ gcc multithreading.c -o multithreading -pthread
codebind@codebind:~$ ./multithreading
Time taken to read the file: 1.929847 seconds
Time taken for thread creation: 0.002063 seconds
Time taken for threads to process: 0.089798 seconds

Top 10 most frequent words:
Word: to, Frequency: 316376
Word: zero, Frequency: 264975
Word: two, Frequency: 192644
Word: was, Frequency: 112807
Word: with, Frequency: 95603
Word: which, Frequency: 54788
Word: were, Frequency: 39086
Word: who, Frequency: 23997
Word: used, Frequency: 22737
Word: when, Frequency: 20623

Total time for the entire program: 0.118316 seconds
codebind@codebind:~$
```

Figure 12: Multithreadind Approach output for (8 threads).

- **Analysis**

This program is designed to process a large text file, count the frequency of each word, and display the top 10 most frequent words. To optimize performance, the program uses multithreading to divide the workload among multiple threads. Each thread processes a portion of the word list, leveraging shared memory to update word frequencies. The goal of this approach is to speed up the word frequency counting process by utilizing multiple CPU cores in parallel.

The program begins by reading a text file into memory. A function (read_file) is used to open the file, read each word, and store it in a dynamically allocated array of strings. Once all the words are loaded into memory, the program sorts the array of words alphabetically using the qsort function.

Thread Creation and Work Division: to accelerate the word frequency counting, the program creates multiple threads. The number of threads can be specified, and each thread processes a specific chunk of the sorted word array. The total word count is divided by the number of threads, ensuring that each thread handles an approximately equal portion of the work. For instance, if there are 8 threads, each thread will handle about 1/8th of the total word list.

In the main function, the program calculates the chunk size for each thread, then creates and starts each thread using pthread_create. Each thread is given a segment of the word list to process, along with a shared frequency array where the results will be stored. Although all threads work in parallel on

different chunks of the word list, they all update the same frequency array in shared memory, which introduces the need for synchronization to avoid race conditions.

The program processes word frequencies by dividing the word list into chunks, with each thread responsible for processing its assigned chunk. Each thread counts the occurrences of words within its chunk, updating a shared frequency array. However, since there is no synchronization, multiple threads could modify the same memory location, which may result in race conditions and incorrect word counts.

After creating the threads, the program uses pthread_join to ensure that all threads complete their tasks before proceeding. Once processing is finished, the frequency array is sorted, and the top 10 most frequent words are displayed. The program also measures the time taken for each stage, from reading the file to sorting the results. Finally, all dynamically allocated memory is freed to avoid memory leaks.

- **Different Number of threads**

Number of threads	**Execution Time**	**Performance**
2 threads	10.344588	0.0966689
4 threads	10.27077	0.0973636
6 threads	10.2113	0.097906
8 threads	10.0208	0.099792

Table 2: Multithreading Approach table.

Why is the output different between the naive and multithreading versions of the program?

The output differs between the naive (single-threaded) and multithreading versions, as shown in our results, due to the presence of race conditions in the multi-threaded approach. In the naive version, the program processes the word list sequentially, ensuring that word frequencies are accurately counted. In the multi-threaded version, multiple threads work on different chunks of the word list simultaneously, all modifying the same shared frequency array. Since the program does not implement synchronization mechanisms (such as mutexes or locks) to manage concurrent access to shared memory, this can lead to race conditions and incorrect frequencies. However, in this particular project, the lack of synchronization is acceptable, as it was not required to implement such mechanisms.

Project Questions

- **Describe your environment: computer specs (Cores? Speed? Memory?), OS, programming language, and IDE tool, and whether you used a virtual machine.**

I worked on this project using a virtual machine running Ubuntu 24.04. My personal computer has an 11th Gen Intel(R) Core (TM) i5-1135G7 processor with 4 cores and 8 logical processors, running at a base speed of 2.40 GHz. The system has 8 GB of memory. I worked in the terminal for coding, using the GCC compiler to compile the C code. I did not use an IDE for this project.

- **How you achieved the multiprocessing and multithreading requirements, i.e. The API and functions that you used.**

In this project, both **multithreading** and **multiprocessing** techniques were implemented to process large word lists efficiently. For multithreading, I used the **POSIX Threads (pthreads)** library in C. The pthread_create() function was used to spawn multiple threads, each processing a chunk of the word list and counting the frequency of each word. The pthread_join() function ensured that the main program waited for all threads to finish before continuing. This approach allows concurrent processing within the same memory space, but the program does not use synchronization mechanisms, making it more prone to race conditions.

For multiprocessing, I used the fork() system call to create child processes. Each child process processes its assigned chunk of the word list, calculates word frequencies, and writes the results to a temporary file. The parent process waits for all child processes to complete using the waitpid() function. Afterward, the program combines the results from the temporary files, calculates the final word frequencies, and prints the top 10 most frequent words. Multiprocessing creates independent memory spaces for each process, which reduces the risk of race conditions compared to multithreading, but requires handling inter-process communication and temporary files.

Both techniques allowed for parallel processing of large datasets, but **multithreading** is generally faster and more efficient in this case, as all threads share the same memory space. However, due to the lack of synchronization in multithreading, the results may be slightly incorrect in terms of word frequencies, as threads can modify shared memory locations concurrently without proper locking mechanisms. In contrast, multiprocessing avoids this issue by working with separate memory spaces for each process.

- **An analysis according to Amdahl's law. What percentage is the serial part of your code? What is the maximum speedup according to the available number of cores? What is the optimal number of child processes or threads?**

For naïve the percentage of serial part is = serial part / total time = (221.149699/224.18758) *100%=98.6%

The serial parts (read from file) are common between naïve, multiprocessing and multithreading and it is almost = 2.35960525sec

Amdahl's Law:

S=1/ (s + p/n)

S: speed

s: serial part

p: parallel part

n: number of process or threads

2 Processes:

Execution Time: 196.78254 sec

Parallel Time: 196.78254−2.35960525=194.42293475 sec

Serial Percentage: (2.35960525/196.78254) ×100=1.20%

S=1/ (2.35960525+(194.42293475/2)) =0.01004 ms−1

---

4 Processes:

Execution Time: 186.3755955 sec

Parallel Time: 186.3755955−2.35960525=184.01599025 sec

Serial Percentage: (2.35960525/186.3755955) ×100=1.27%

S=1/(2.35960525+(184.01599025/4)) =0.02067 ms−1

---

6 Processes:

Execution Time: 138.347824 sec

Parallel Time: 138.347824−2.35960525=135.98821875 sec

Serial Percentage: (2.35960525/138.347824) ×100=1.71%

S=1/ (2.35960525+(135.98821875/6) =0.03996 ms−1

---

8 Processes:

Execution Time: 185.01216 sec

Parallel Time: 185.01216−2.35960525=182.65255475 sec

Serial Percentage: (2.35960525/185.01216) ×100=1.27%

S=1/ (2.35960525+(182.65255475/8)) =0.03971 ms−1

---

2 Threads:

Execution Time: 10.344588 sec

Parallel Time: 10.344588−2.35960525=7.98498275 sec

Serial Percentage: (2.35960525/10.344588) ×100=22.81%

S=1/ (2.35960525+(7.98498275/2)) =ms−1

---

4 Threads:

Execution Time: 10.27077 sec

Parallel Time: 10.27077−2.35960525=7.91116475 sec

Serial Percentage: (2.35960525/10.27077) ×100=22.97%

S=1/ (2.35960525+(7.91116475/4) =0.2305 ms−1

---

6 Threads:

Execution Time: 10.2113 sec

Parallel Time: 10.2113−2.35960525=.85169475 sec

Serial Percentage: (2.35960525/10.2113) ×100=23.11%

S=1/ (2.35960525+(7.85169475/6)) = ms−1

---

8 Threads:

Execution Time: 10.0208 sec

Parallel Time: 10.0208−2.35960525=7.66119475 sec

Serial Percentage: (2.35960525/10.0208) ×100=23.55%

S=1/ (2.35960525+(7.66119475/8)) =0.3015 ms−1

The optimal number of child processes is **6**, as it achieves the lowest execution time (138.34 seconds) compared to 2, 4, or 8 processes; adding more than 6 processes introduces overhead, leading to worse performance. For threads, the optimal number is **8**, with the lowest execution time of 10.02 seconds, as threads continue to scale better in this case without significant overhead. Therefore, for large-scale tasks involving processes, 6 is optimal, while for smaller-scale or thread-focused tasks, 8 threads provide the best performance.

- **A table that compares the performance of the 3 approaches.**

Approach	Execution Time	Performance
naive	224.18758sec	4.521843ms-1
2 processes	196.78254sec	5.08175166ms-1
4 processes	186.3755955sec	5.365498 ms-1
6 processes	138.347824sec	7.228155 ms-1
8 processes	185.01216sec	5.40505 ms-1
2 threads	10.344588sec	0.0966689
4 threads	10.27077sec	0.0973636
6 threads	10.2113sec	0.097906
8 threads	10.0208sec	0.099792

Table 3: Three Approaches table.

- **Comment on the differences in performance**

The performance differences between the naive approach, multiprocessing, and multithreading highlight the impact of parallelization. The naive approach, being entirely serial, has the highest execution time (224.19 seconds) and does not utilize available cores, with 98.6% of the code being serial. Multiprocessing reduces execution time significantly by leveraging multiple CPU cores, achieving the best performance with 6 processes (138.35 seconds) due to an optimal balance of workload and process management. However, adding more processes beyond this point results in diminishing returns due to overhead. Multithreading delivers the fastest execution time (10.02 seconds with 8 threads) but offers marginal improvements with additional threads because of memory access contention and synchronization overhead. While multiprocessing is optimal for CPU-intensive tasks, multithreading benefits from reduced overhead and efficient memory sharing, making it the fastest overall. For this workload, multiprocessing with 6 processes provides the best balance, while multithreading with 8 threads achieves the shortest execution time

Conclusion

In conclusion, this project shows how using parallel computing techniques like multiprocessing and multithreading can greatly improve the performance of programs that handle large amounts of data. The naive approach, which runs everything sequentially, takes the longest time and doesn't scale well. Multiprocessing makes good use of multiple CPU cores, with six processes being the most efficient balance. On the other hand, multithreading achieves the fastest results with eight threads, as it reduces overhead. This project highlights how understanding the task and system can help choose the right parallel approach to save time and improve efficiency. It demonstrates the power of parallel computing to make programs faster and more scalable.